Strands Agents 기반 EKS 최적화 보고서 개발기

Strands Agents | MCP Server | Agent Skills | Langfuse

SPEAKER

발표자 소개

한승호

클라우드 엔지니어 · LG 그룹사 퍼블릭 클라우드 관리 · EKS 50개 이상 클러스터 운영 중

- CloudNet@ 스터디 참여 — AEWS, T101, KANS 등..
- 오픈소스 · 신기술에 관심이 많고, 특히 자동화에 관심이 많습니다!
- https://hanhorang.tistory.com

AGENDA

다룰 내용

배경
WHY

01
배경 및 문제 정의
EKS 운영 복잡성 · 인수인계 비용

1차 프로젝트
맨땅 헤딩

02
초기 설계 및 구현
MCP Server + Strands Agent 결합

03
운영 적용의 한계
토큰 한계 · 도구 과잉 · 정확도 편차

2차 프로젝트
개선

04
개선 접근 방법
Agent Skill · Langfuse

05
그럼에도 불구하고
정확도 향상과 토큰 절약을 위한 팁 공유

마무리
INSIGHT

06
정리
정리 · 트렌드/전망

EKS 운영 & 관리의 어려움

01 **복잡한 운영 환경** - 다양한 AWS 리소스와 Add-on 조합 - 서비스별 상이한 운영 구조

02 **파편화된 운영 가이드** - 너무 많은 문서와 Best Practice - 환경별 기준 판단이 어려움

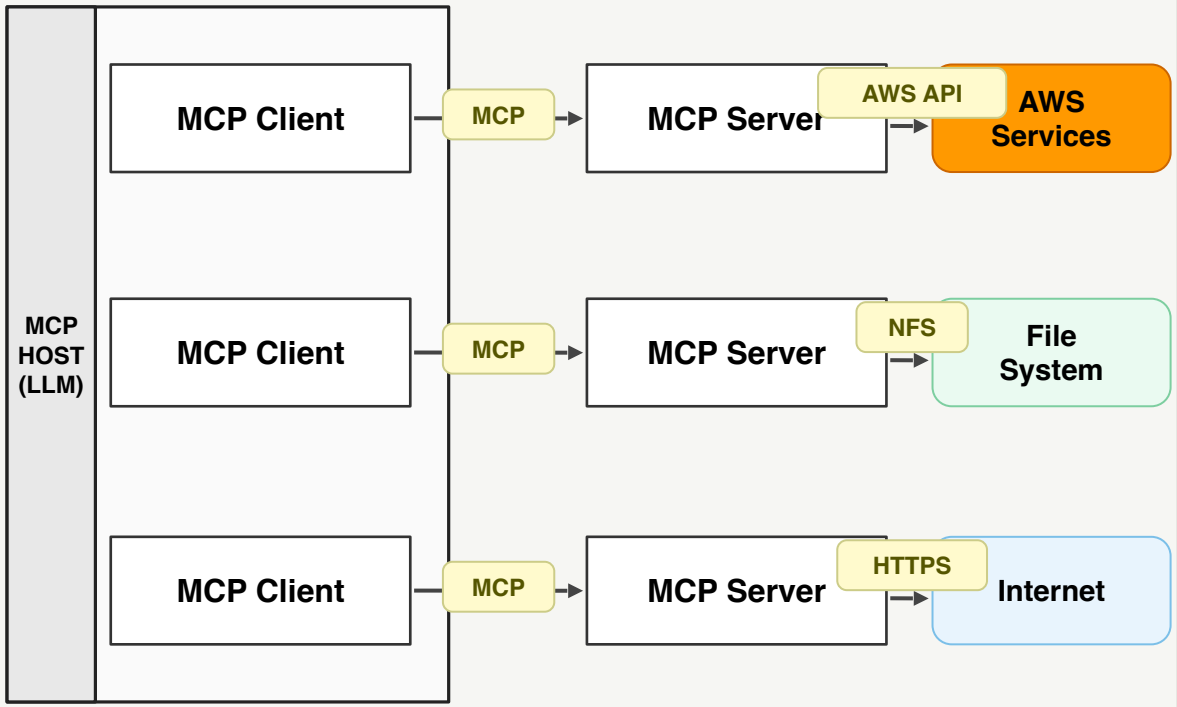
03 **반복되는 운영 점검** - EOS / 비용 / 보안 / 성능 확인 반복 - 클러스터 수만큼 반복 작업 필요

AI가 운영 데이터를 자동으로 분석하고 인사이트로 제공해준다면 ?

기술 소개(1/2) — MCP & Strands Agents

MCP (Model Context Protocol)

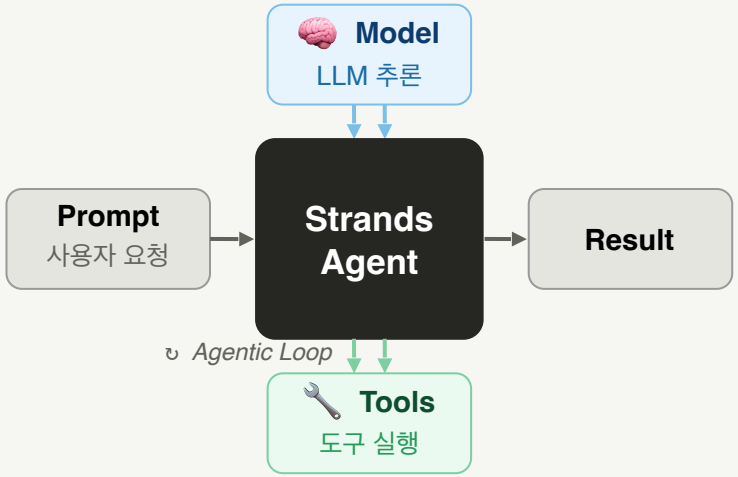
AI가 외부 데이터와 도구에 쉽게 접근할 수 있도록
표준화된 연결 방식을 제공하는 프로토콜



USB-C 포트처럼 — 연결하면 확장성있게 동작

Strands Agents

모델 · 프롬프트 · 도구만 정의하면
에이전트가 알아서 오케스트레이션하는 오픈소스 SDK



기존 프레임워크 대비 복잡한 오케스트레이션 로직 불필요

Tools에 MCP 서버을 연동하여 수천 개 도구 자동 활용 가능

기술 소개(2/2) - CFM Tips MCP Server

CFM Tips란?

AWS 클라우드 재무 관리 기술 구현 플레이북(CFM TIP)
- AWS 비용 분석 및 최적화 권장사항을 제공하는 단계별 플레이북

오픈소스 CFM Tips MCP Server

github.com/aws-samples/sample-cfm-tips-mcp

AWS 각 자원에 대한 최적화 항목을 플레이북 형태로 제공

Cost Optimization Playbooks

- 🔧 **EC2 Right Sizing** - Identify underutilized EC2 instances with 12 specialized analysis tools
- 💾 **EBS Optimization** - Find unused and underutilized volumes
- 🗄️ **RDS Optimization** - Identify idle and underutilized databases
- ⚡ **Lambda Optimization** - Find overprovisioned and unused functions
- 🪣 **S3 Optimization** - Comprehensive S3 cost analysis and storage class optimization with 11 specialized tools
- 📋 **CloudTrail Optimization** - Analyze and optimize CloudTrail configurations
- 📊 **CloudWatch Optimization** - Optimize monitoring costs across logs, metrics, alarms, and dashboards
- 💰 **Database Savings Plans** - Analyze and optimize database commitment plans for Aurora, RDS, DynamoDB, and more
- 🌐 **NAT Gateway Optimization** - Identify underutilized, redundant, and unused NAT Gateways
- 📈 **Comprehensive Analysis** - Multi-service cost analysis

EKS 최적화 항목 추가 개발

오픈소스 CFM Tips MCP Server에는 EKS 최적화 미포함
→ 바이브코딩(Kiro)으로 EKS 플레이북 항목 추가

eks_cluster_analysis

클러스터 버전 · EOS 경고 · Container Insights 확인

eks_pod_rightsizing

Container Insights 기반 CPU/Memory 사용률 분석

eks_node_optimization

Graviton 전환 · Spot 기회 · 구세대 업그레이드 권장

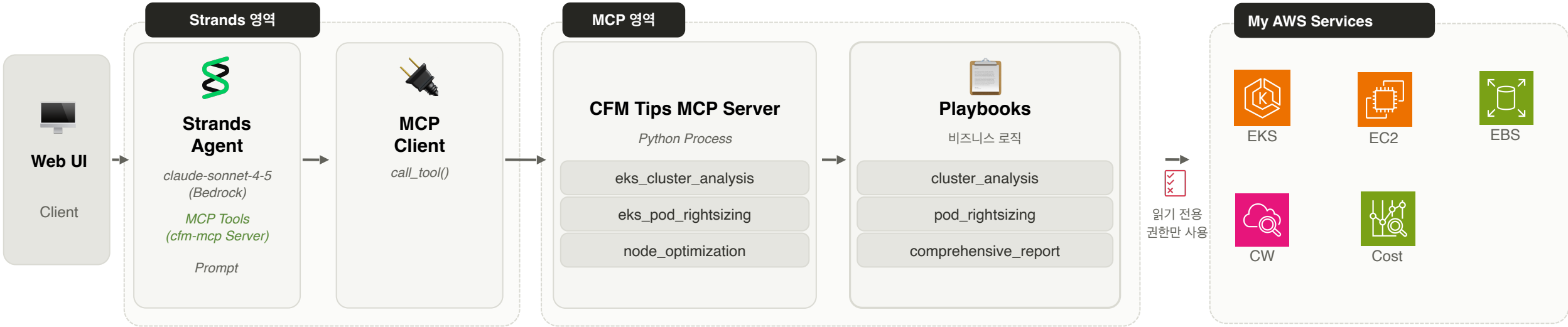
eks_storage_optimization

PVC EBS 사용률 · gp2→gp3 마이그레이션 권장

eks_comprehensive_report

전체 분석 통합 종합 리포트 생성

초기 아키텍처



IAM 최소 권한 원칙

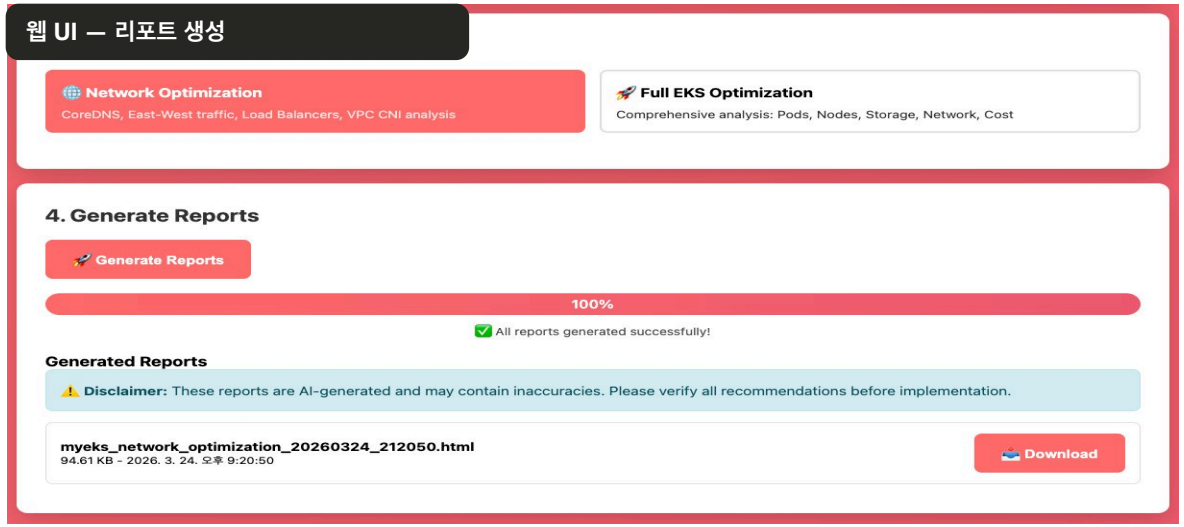
- MCP Server에 부여된 기본 IAM Role만 사용 — 별도 자격 증명 불필요
- 읽기 전용 권한만 부여 — 리소스 생성·수정·삭제 없음
- 자격 증명 코드 내 미포함 — 보안 위험 최소화

기능 확장에 따른 추가 개발 항목

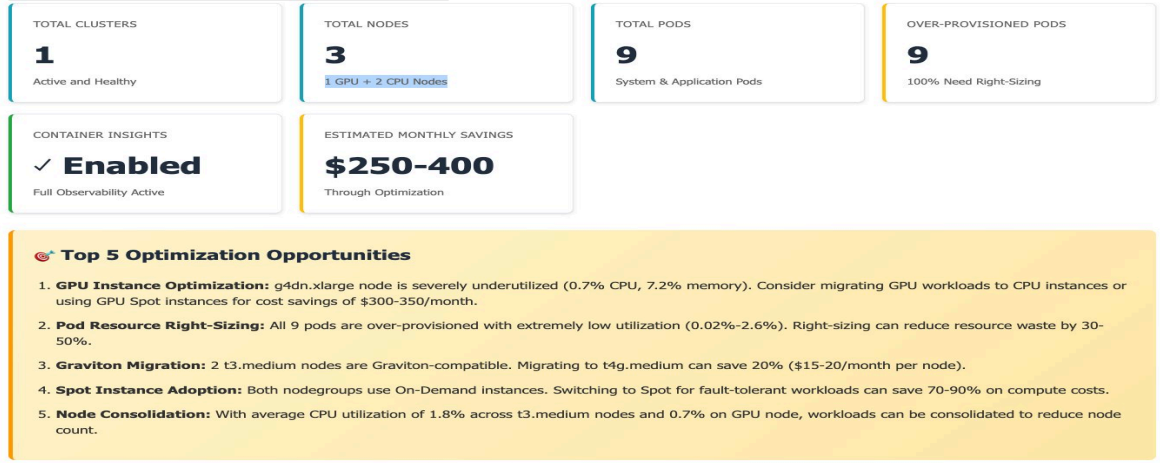
EKS Playbook 추가	CFM Tips에 없는 EKS 최적화 로직 직접 작성 (Boto3 기반)
커스텀 프롬프트	분석 기준 · 출력 포맷 · 권고 기준을 프롬프트로 정의
Web UI (Streamlit)	클러스터 선택 → 보고서 타입 선택 → 생성 버튼 인터페이스

구현 결과

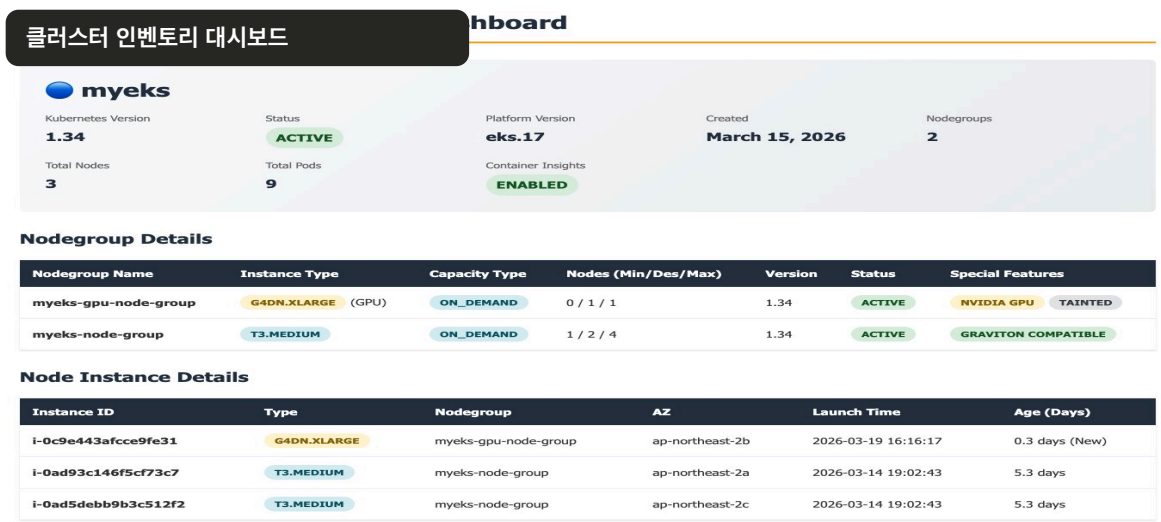
웹 UI — 리포트 생성



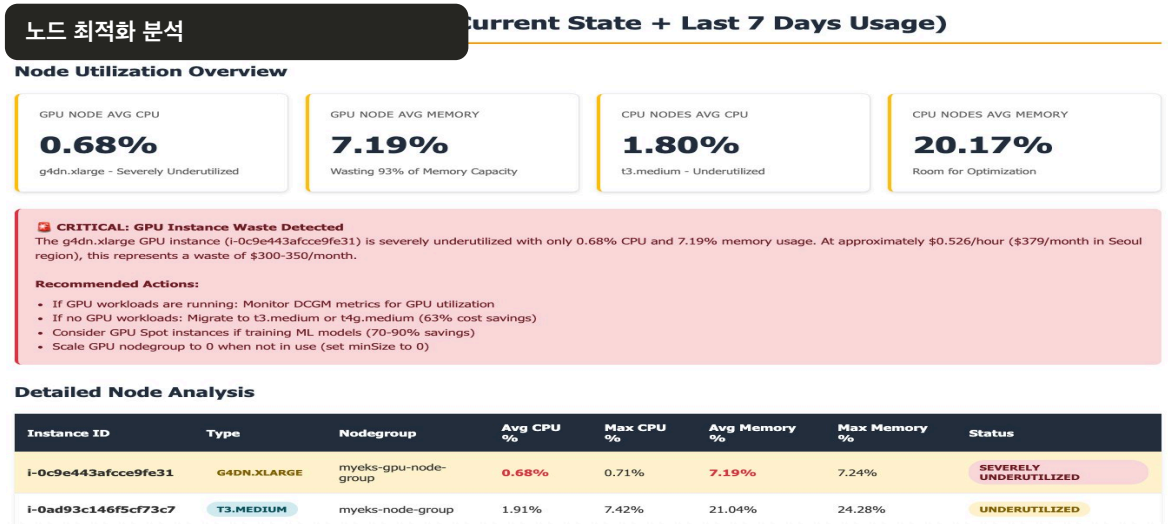
Executive Summary



클러스터 인벤토리 대시보드



노드 최적화 분석



1차 프로젝트의 한계

구조 문제

기능 확장에 따라 Agent가 감당하기 어려워졌다

기능 확장

- 네트워크 최적화
- 노드 스케일링 최적화
- 보안 분석

↓

단일 Agent가 모든 역할 수행

- 도구 분리 & 프롬프트 구별 지침을 했지만 한계 도달

↓

결국...

- 컨텍스트가 과도하게 커짐
- Tool 선택 정확도 저하
- 결과 일관성 저하
- 검증 비용 증가

운영 문제

운영 적용의 애로사항

01 결과 신뢰성

- 동일 요청에도 매번 결과가 달라짐
- 클러스터 상태와 맞지 않는 권고 발생(ex. 비용)

02 검증 비용

- AI 결과를 매번 사람이 검토
- 자동화했지만 오히려 검증 시간이 증가

03 HTML 확장자의 애로사항

- HTML 확장자 보안 및 수정하기 어려움

04 재사용성

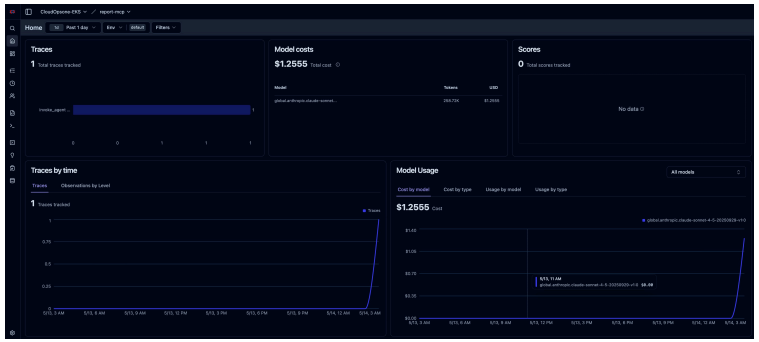
- 특정 환경에 의존적인 구조
- 다른 클러스터에서 동작 보장 어려움

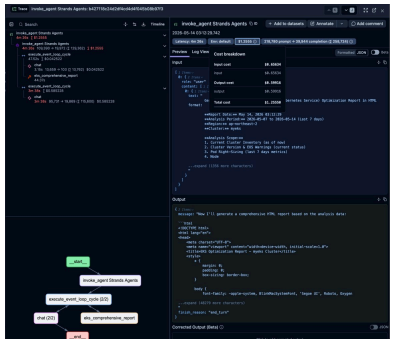
무엇을 도입했나 — Langfuse & Agent Skill

1차 한계 토큰 폭증 · Tool 선택 실패 · 결과 불일치 · 검증 비용 증가 → **Langfuse** + **Agent Skill** 로 개선

Langfuse

비용 · 토큰 · 레이턴시를 한눈에 — 이슈 수치화





- 토큰 사용량과 비용을 호출 단위로 트레이싱 가능
- 어느 단계에서 토큰이 터지는지 파악 가능
- 성능 병목 구간 시각화 → 개선 우선순위 도출

Agent Skill

재사용 가능한 모듈 단위 설계

기존 방식 vs Skill 방식 — 컨텍스트 윈도우 비교



- 컨텍스트 크기 감소 → 토큰 절약
- Skill 재사용 → 지침&스크립트에 따른 결과로 일관성 확보 및 적용 확대 가능
- 기능 조합 : Skills를 결합하여 복잡한 워크플로우 구축 가능

토큰 사용량 초과	→	Skill 분리 → 컨텍스트 격리
Tool 선택 실패	→	Description 명확화 → 정확도 향상
결과불일치·검증비용	→	Skill 재사용 + Validation → 일관성 & 자동 검증

Agent Skill 활용 — 재사용 가능한 AI 모듈

💡 **Claude Skill 활용 가능 — 오픈소스 Skill 활용 가능**

EKS Operation Review Skill

github.com/aws-samples/sample-eks-operation-review-skill

전체 파일 구조

```
skills/eks-operation-review/
├── SKILL.md                          ← 스킬 진입점 (메타데이터 + 지침)
├── references/                       ← Phase 3 리소스 파일들
│   ├── cluster-lifecycle.md                (섹션 01)
│   ├── infrastructure-as-code.md           (섹션 02)
│   ├── access-identity.md                  (섹션 03)
│   ├── observability.md                    (섹션 04)
│   ├── workload-configuration.md           (섹션 05)
│   ├── networking.md                       (섹션 06)
│   ├── autoscaling.md                      (섹션 07)
│   ├── deployment-practices.md             (섹션 08)
│   ├── operational-processes.md            (섹션 09)
│   ├── addon-management.md                 (섹션 10)
│   └── report-generation.md                HTML 결과 변환 지침
```

SKILL.md

```yaml
---
name: eks-operation-review
description: "EKS Operational Excellence
  assessment. Use when asked to review
  an EKS cluster..."
---
```

→ *LLM*이 스킬의 존재와 활용 시점만 인식

References (각 지침 파일 세분화)

What Gets Assessed

#	Area	Examples
01	Cluster Lifecycle	Version currency, upgrade readiness, deprecated APIs
02	Infrastructure as Code	IaC provenance, GitOps tools, drift detection
03	Access & Identity	IRSA / Pod Identity, RBAC, API server endpoint, Pod Security Admission
04	Observability	Control plane logging, metrics, log aggregation, alerting
05	Workload Configuration	Resource requests, health probes, PDBs, image tags
06	Networking	IP capacity, CoreDNS, network policies
07	Autoscaling	Karpenter / CA, HPA, topology spread
08	Deployment Practices	Rollout strategy, CI/CD, graceful shutdown
09	Operational Processes	Backup / DR, runbooks, on-call
10	Add-on Management	Managed add-ons, node health monitoring, cluster insights

Agent Skill 활용 — 재사용 가능한 AI 모듈

💡 **Claude Skill 활용 가능 — 오픈소스 Skill 활용 가능**

EKS Operation Review Skill

지침 파일 (cluster-lifecycle.md)

Checks to Execute

1.1 — Control Plane Version Currency

What to check:

- Describe the cluster to get the current Kubernetes version and platform version
- Look up the version in the calendar above to determine actual support status
- Report the actual support status, NOT the `supportType` API field

How to check:

1. Describe the cluster → get `version` and `platformVersion`
2. Match the version against the calendar table above
3. Report: version, standard/extended status, and when current support period ends

Version out-of-range guard: If the cluster version is higher than the highest version in this table (currently 1.35), rate GREEN — a newer version is by definition on standard support. Add a note: "Version 1.X is newer than the reference table (last verified YYYY-MM-DD). Rated GREEN as latest. Update the table when convenient."

Rating:

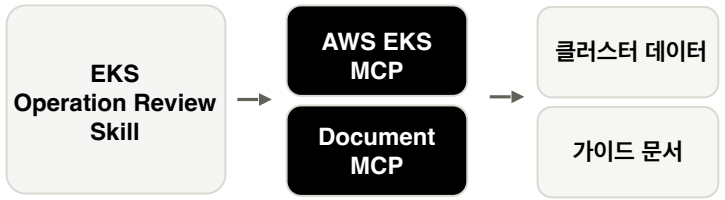
- 🟢 GREEN: On v1.35 or v1.34 (latest or N-1, standard support), OR version newer than table
- 🟡 AMBER: On v1.33 or v1.32 (standard support but older)
- 🔴 RED: On v1.31 or below (extended support — paying $0.60/hr vs $0.10/hr, 6x premium)
- ⬜ UNKNOWN: Cannot determine (should not happen with live access)

Key talking point: Extended support costs $0.60/hr vs $0.10/hr — that's ~$4,300/month extra per cluster.

10개의 점검 영역 / 38개의 점검 항목 기준 지침 안내

데이터 수집 — AWS MCP 서버 연동

실제 클러스터 데이터와 운영 가이드는 AWS EKS · Document 서버를 통해 수집



AWS MCP Server	역할
EKS MCP Server	클러스터 운영 상태 및 리소스 정보 수집
Document MCP Server	AWS 공식 가이드 및 Best Practice 근거 제공

Skill 결과





Agent Skill 활용 — 재사용 가능한 AI 모듈

💡 **Claude Plugin 활용 가능 — 오픈소스 Plugin 활용 가능**

PPT 생성 Plugin (McKinsey PPTX)

github.com/seulee26/mckinsey-pptx

```
mckinsey-pptx/
├── .claude-plugin/plugin.json  ← Claude Code 플러그인 메타데이터
├── commands/mckinsey-deck.md  ← /mckinsey-deck 슬래시 커맨드 진입점
├── agents/mckinsey-slide-agent.md  ← AI 에이전트 · 9단계 워크플로우
├── mckinsey_pptx/agent/CATALOG.md  ← 40개 슬라이드 템플릿 카탈로그
├── mckinsey_pptx/  ← Python 패키지 · PPTX 생성 담당
├── _input/  ← 에이전트 참조 데이터 / JSON
└── _output/  ← 생성된 PPTX + 빌드 스크립트
```

Plugin 실행 흐름

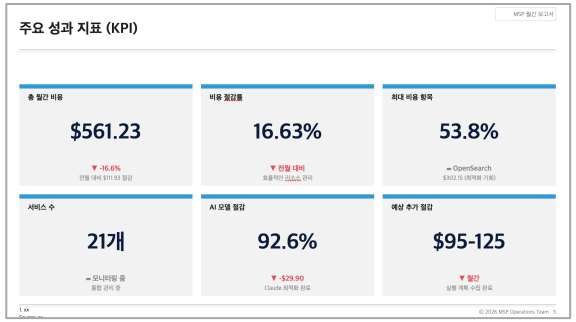


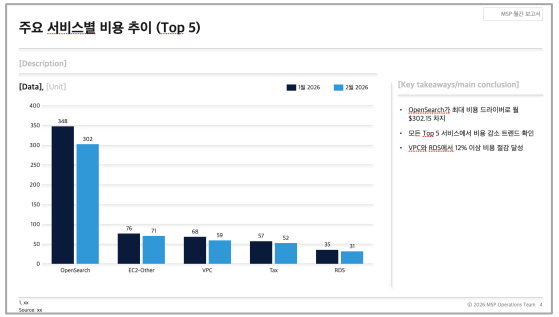
Prompt (스킬 사용 예제 프롬프트)

폴더 안에 있는 json 파일을 기반으로 월간보고서 PPT로 만들어주세요.

PPT 결과









Claude Skill 에서 Strands Agent Skill 로 변환

Sample-strands-agents-agentskills

github.com/aws-samples/sample-strands-agents-agentskills

Strands Agent SDK에서 에이전트 스킬을 사용하기 위한 기본 아키텍처

안정성, 통제력, 확장성을 가진 AWS 클라우드 네이티브 호환 스킬로 전환

01 보안 및 거버넌스

Private LLM(Bedrock) 활용으로 데이터 유출 원천 차단
Bedrock 가드레일로 오남용 방지

02 통합 관측성 제공

Langfuse 또는 AgentCore 호환으로 트레이스 관측 가능

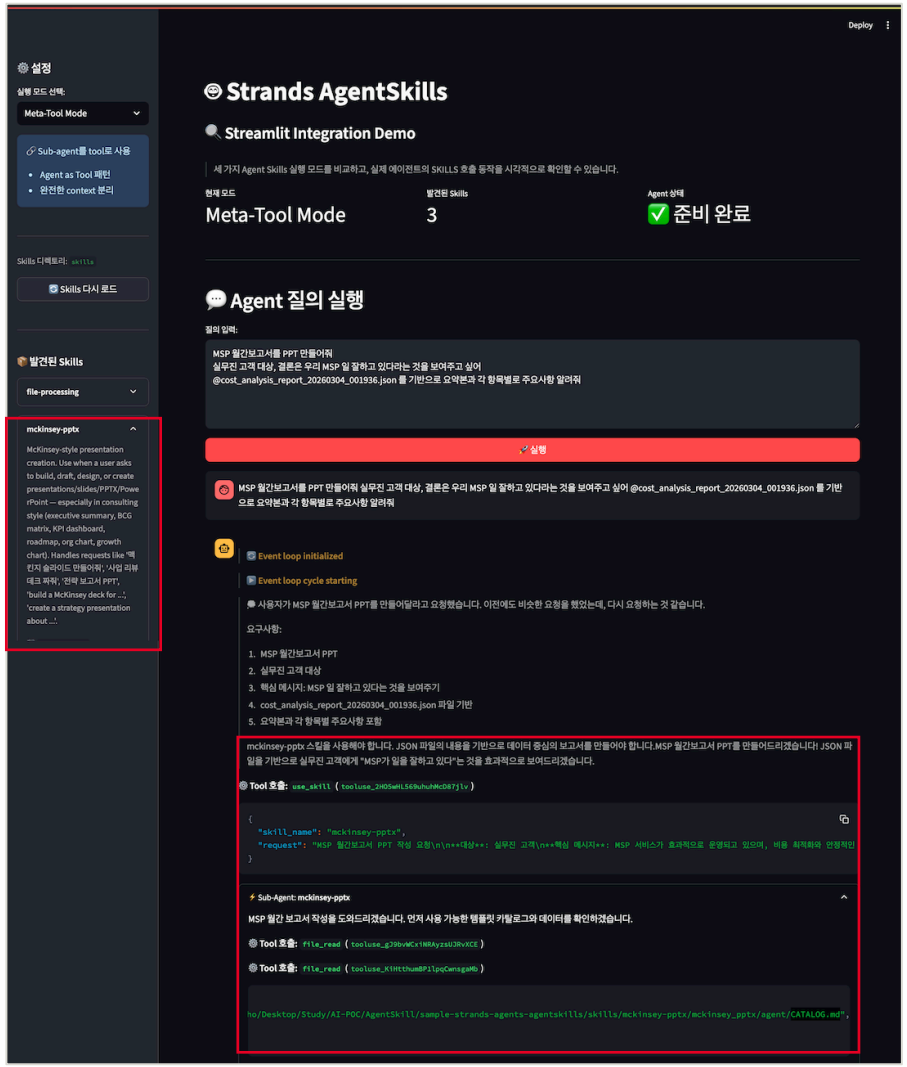
03 복잡한 워크플로우 작성 가능

Strands Agent 기반 Graph 설계로 사용자 정의 워크플로우 작성 가능
AWS Lambda · EventBridge 연동으로 이벤트 기반 자동 실행 가능

04 엔터프라이즈급 확장성

AWS AgentCore 메모리 등 프로덕션 배포 기능 사용 가능

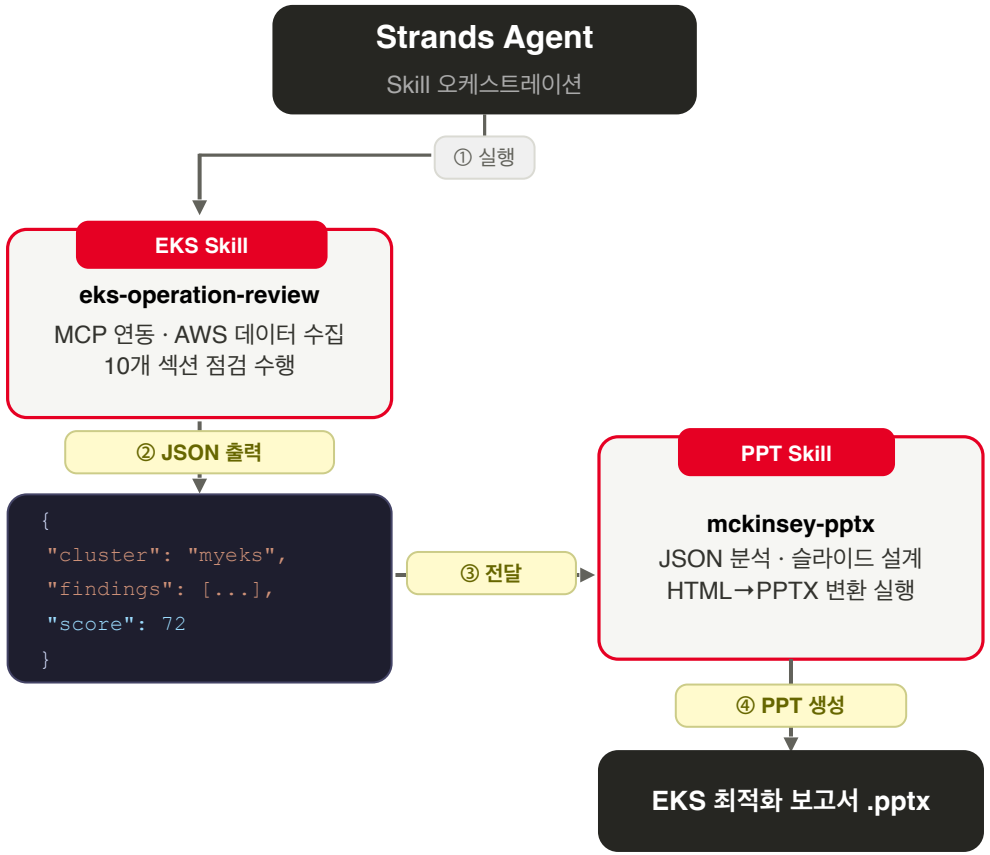
스킬 구현 : 점진적 컨텐츠 로드



Skill 재사용 — EKS 점검 → PPT 자동 생성

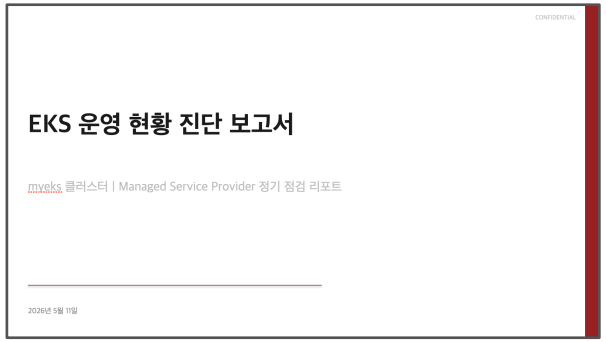
두 Skill을 조합한 파이프라인

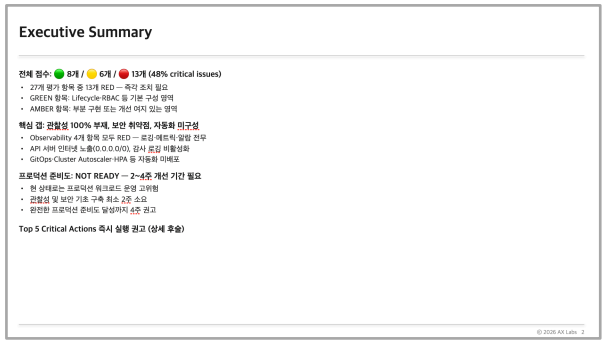
EKS 점검 결과를 JSON으로 받아 PPT 보고서로 자동 변환

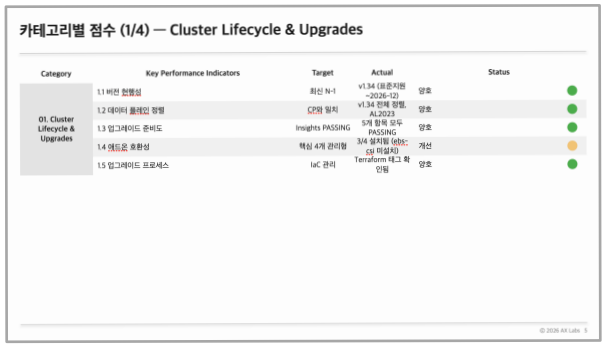


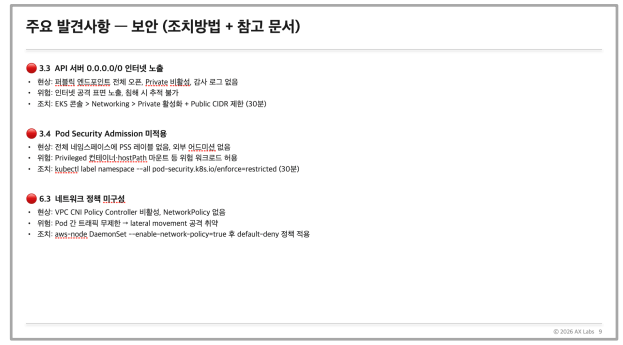
실제 결과 화면

EKS 점검 JSON → PPT 자동 생성 결과









행복회로의 결말 : 무너진 일관성과 토큰 폭탄

무너진 일관성

결과 일관성 58% 동일



동일 클러스터 2회 연속 실행 결과 비교 (maturity_score 상이)

3.1 Pod 수준 AWS 권한 (IRSA / EKS Pod Identity)

Secrets · Pod 환경변수에 AWS 자격증명 하드코딩 여부 — 발견 여부를 LLM이 정성 판단

Run 1	"credentials hardcoded as env vars" 발견	RED
Run 2	*"No hardcoded credentials detected"*	AMBER

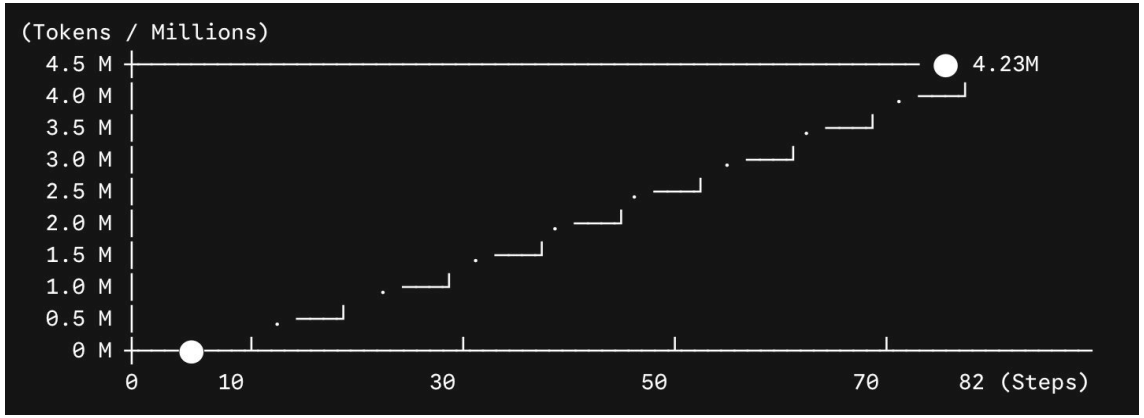
3.2 최소 권한 RBAC — 누가 클러스터에 접근할 수 있는가

cluster-admin 바인딩 수, 와일드카드 권한 여부 — '몇 개가 적정한가'를 LLM이 정성 판단

Run 1	EKS managed users만 확인, 추가 계정 미발견	GREEN
Run 2	사용자 계정 추가 발견	AMBER

토큰 폭탄

1회 최대 400만 토큰 사용



왜 루프마다 토큰이 쌓이는가

①

컨텍스트 누적

한 번 읽은 references
12개 + AWS 결과가
모든 LLM 호출에 반복 포함

②

실패 재시도 누적

MCP 명령어 오류 →
실패 기록도 컨텍스트 추가
재시도마다 토큰 손실 가중

③

판단 단계 토큰 소비 집중

38개 항목 종합 판단 시
토큰 소비 폭발적 증가
수집보다 판단에서 압도적

일관성 개선 시도

Temperature 조정과 멀티 에이전트 구성을 시도했지만 근본 원인을 해결하지 못했습니다

01 Temperature 조정

효과 없음

시도 temperature = 0으로 고정

LLM의 확률적 출력을 줄여 동일 입력에서 동일 결과를 기대

결과 동일 입력에도 다른 판단 여전히 발생

temperature 0은 표현의 다양성만 줄일 뿐

판단 기준 자체가 모호하면 모델은 그 자유도를 그대로 사용함

02 지침별 멀티 에이전트 구성

효과 없음

시도 각 지침마다 독립 에이전트 분리 운용

컨텍스트 오염 방지 목적으로 에이전트를 지침 단위로 완전 분리

결과 판단 편차 해소되지 않음

에이전트를 100개로 나눠도 기준이 모호하면

100개의 오류가 생길 뿐 — 구조는 해결책이 아니었음

원인 분석

모델 및 구조가 아니라 판단 기준의 모호성임

판단 기준에 OR 조건·추상명사가 열려 있으면
LLM은 매번 다른 해석을 내림

일관성 개선

판단 기준을 LLM이 해석할 수 없도록 명시적 조건으로 재작성 · 유효성 검증 함수 추가

01 판단 기준 모호성 분석

명확한 항목 (결정론적)

1.1 Control Plane Version

캘린더 테이블 lookup → 결정론적 판단
drift 거의 없음

모호한 항목 (정성적 판단)

1.3 Upgrade Readiness

AMBER: WARNING-level insights,
or no automated detection tooling

→ "or" — 한쪽만 충족 시 어디로?

1.5 Upgrade Maturity

GREEN: evidence of IaC management
AMBER: unclear upgrade process

→ "evidence"·"unclear" 정의 없음

같은 데이터에도 다른 판단이 나올 여지가 명시적으로 열려 있음

02 기준 모호성 제거

BEFORE — 모호한 조건

GREEN	*"Cluster has environment tags, evidence of IaC management"*
AMBER	*"No environment tags, or unclear upgrade process"*

▼ 명시적 조건으로 재작성

AFTER — 명시적 조건

GREEN	tag "Environment" AND tag "ManagedBy" AND [“terraform","eksctl","cloudformation"] OR CloudFormation stack으로 생성
AMBER	tag "Environment" 존재 AND IaC tag 없음
RED	tag "Environment" 없음

03 유효성 검증 추가

항목 카운팅 정확도 보장

문제

상태 개수 합계가 전체 항목 수와 일치하는지 검증하는 로직 부재 → 결과 누락·중복 가능성 존재

해결

집계 후 총합 검증 함수 추가

✓ 상태별 카운팅

GREEN/AMBER/RED/UNKNOWN 각 집계

✓ 합계 검증

총합 = 전체 항목 수 확인

✓ 조기 경보

불일치 시 리포트 전 오류 반환

토큰 절감 — 접근 방법과 시도

토큰 절감을 위한 Context Window 처리 기법 적용

시도 방법

프롬프트 토큰 제약

시도

SKILL.md에 Context Budget Rules 추가, 필드 필터링 지시

결과

LLM이 지시를 따라 검증용 데이터 수집 스킵 → 일관성 저하

멀티 에이전트 구성

시도

지침별 독립 에이전트 분리, Context 격리로 토큰 절감 기대

결과

각 에이전트가 동일한 tool 호출을 독립 반복 수행
검증 토큰 비용 증가

캐싱 활성화 — 효과 있음

시도

메세지 캐싱 적용

결과

단일 실행 기준 토큰 72% 절약과 캐시 비용에 따라 비용 감소

계층	접근 방식	핵심 원리	실패하는 경우 (다음 계층이 보완)
Layer 1: 아키텍처	멀티 에이전트 구조를 통해 Context 격리하기	큰 작업을 여러 에이전트로 분리, 각 에이전트가 독립적 Context 사용	격리만으로 부족한 경우 (→ Layer2로 보완)
Layer 2: 프롬프트	프롬프트로 Context 유입량 제어하기	행동 규칙 지정 및 출력 제어	프롬프트로도 부족한 경우 (→ Layer3로 보완)
Layer 3: 도구	파일 기반 Context 외부화 ("Context에는 포인터만, 데이터는 파일에")	무거운 컨텐츠는 컨텍스트에 의존하지 않고 파일로 분리하여 유지	그 외 문제가 발생하는 경우 (→ Layer4)
Layer 4: 검증/안전장치	검증 에이전트와 안전장치로 품질 보증하기	오류를 잡고 Context overflow를 방지하는 독립적 장치 구현	최종 방어선

https://aws.amazon.com/ko/blogs/tech/context-engineering-from-deep-insight/

모델 사용 유형 토큰별 비용 : 캐시 읽기 토큰의 경우 최대 1/10 절감 가능

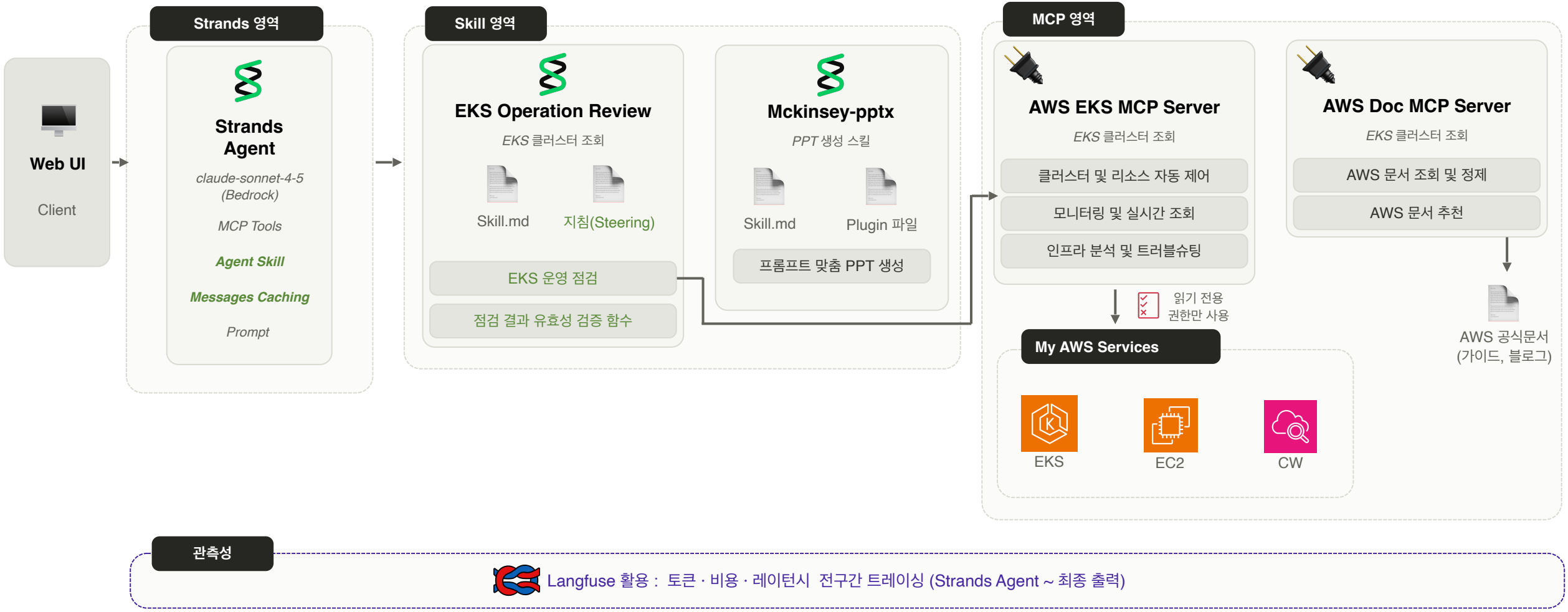
Anthropic 온디맨드 요금

Bedrock us-east-1 (Price per 1M tokens)

모델	Input	Output	Batch In	Batch Out	Cache Write	Cache Read
Claude Opus 4.6	$5.00	$25.00	$2.50	$12.50	$6.25	$0.50
Claude Haiku 4.5	$1.00	$5.00	$0.50	$2.50	$1.25	$0.10
Claude Sonnet 4.5	$3.00	$15.00	$1.50	$7.50	$3.75	$0.30

※ AWS Bedrock 공식 가격 기준 · 현재 프로젝트: Claude Sonnet 4.5 사용

최종 아키텍처



관측성

Langfuse 활용 : 토큰 · 비용 · 레이턴시 전구간 트레이싱 (Strands Agent ~ 최종 출력)

개선 결과

일관성 개선 — 일관성 100% 동일 결과 확인

_output > {} EKS-Operation-Review-myeks-2026-05-20-2205.json > ...


```
{
  "metadata": {
    "cluster_name": "myeks",
    "region": "ap-northeast-2",
    "kubernetes_version": "1.34",
    "platform_version": "eks.23",
    "aws_account_id": "-",
    "authentication_mode": "API_AND_CONFIG_MAP",
    "assessment_date": "2026-05-20",
    "assessment_time": "22:05"
  },
  "maturity_score": {
    "green": 7,
    "amber": 11,
    "red": 16,
    "unknown": 4,
    "total_rated": 34,
    "green_pct": 20.6,
    "amber_pct": 32.4,
    "red_pct": 47.1
  },
  "executive_summary": "The myeks cluster (Kubernetes 1.34) demonstrates several operational str
  "findings": [
    {
      "section_id": "01",
      "section_name": "Cluster Lifecycle & Upgrades",
      "items": [
        {
          "id": "1.1",
          "name": "Control Plane Version Currency",
          "status": "GREEN",
          "current_state": "Cluster is running Kubernetes 1.34 (platform version eks.23), which
          "recommendation": "",
          "references": [
            "https://docs.aws.amazon.com/eks/latest/userguide/kubernetes-versions.html",
            "https://docs.aws.amazon.com/eks/latest/userguide/platform-versions.html"
          ]
        },
```

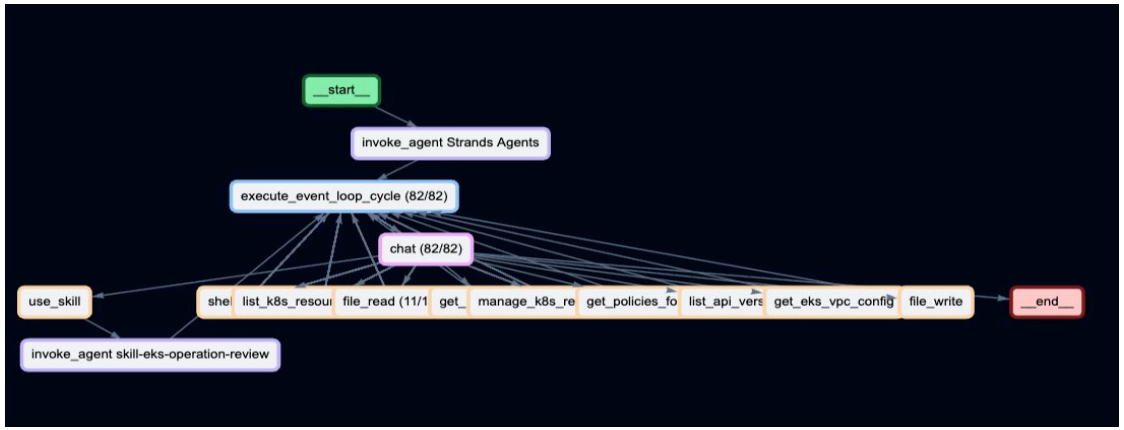

_output > {} EKS-Operation-Review-myeks-2026-05-20-2235.json > ...


```
{
  "metadata": {
    "cluster_name": "myeks",
    "region": "ap-northeast-2",
    "kubernetes_version": "1.34",
    "platform_version": "eks.23",
    "aws_account_id": "-",
    "authentication_mode": "API_AND_CONFIG_MAP",
    "assessment_date": "2026-05-20",
    "assessment_time": "22:35"
  },
  "maturity_score": {
    "green": 7,
    "amber": 11,
    "red": 16,
    "unknown": 4,
    "total_rated": 34,
    "green_pct": 20.6,
    "amber_pct": 32.4,
    "red_pct": 47.1
  },
  "executive_summary": "The myeks cluster (v1.34) demonstrates several operational strengths: it is prov
  "findings": [
    {
      "section_id": "01",
      "section_name": "Cluster Lifecycle & Upgrades",
      "items": [
        {
          "id": "1.1",
          "name": "Control Plane Version Currency",
          "status": "GREEN",
          "current_state": "Cluster is running Kubernetes 1.34 (platform version eks.23), which is in st
          "recommendation": "",
          "references": [
            "https://docs.aws.amazon.com/eks/latest/userguide/kubernetes-versions.html",
            "https://docs.aws.amazon.com/eks/latest/userguide/platform-versions.html"
          ]
        },
```

개선 결과

토큰 절약 — 일관성 100% 동일 결과 확인

BEFORE 캐싱 미적용

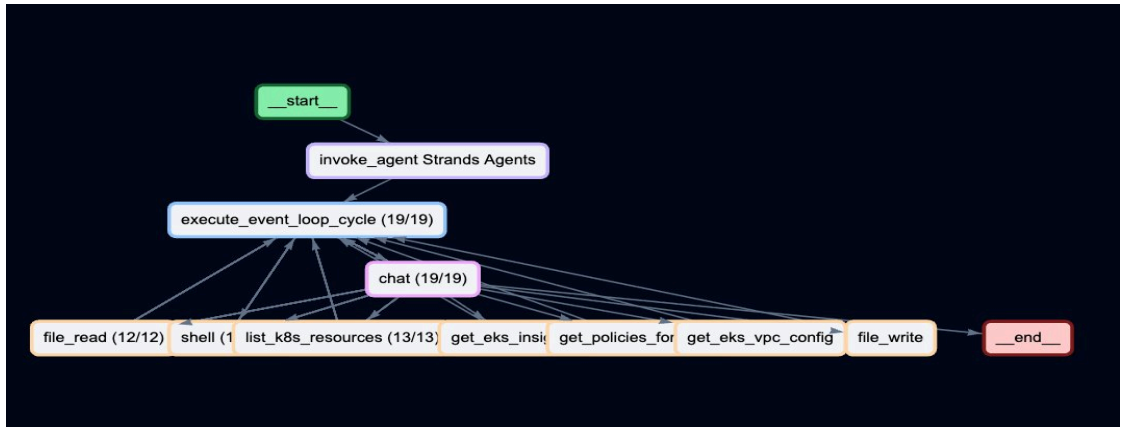


Usage breakdown

Input usage	**11,017,334**
input	8,377,814
cache_read_input_tokens	2,639,520
cache_write_input_tokens	0
Output usage	**92,667**
Total usage	**11,110,001**

Total **11,110,001** tokens

AFTER Message Caching 적용



Usage breakdown

Input usage	**2,952,422**
cache_read_input_tokens	2,691,596
cache_write_input_tokens	228,932
input	31,894
Output usage	**79,094**
Total usage	**3,031,516**

Total **3,031,516** tokens

이게 되네, 왜 되는 거지, 이게 맞는 건가?

이게 되네

바이브코딩 체감

- Strands 에이전트 개발, 프로젝트 이관 편이

왜 되는 거지?

데이터 수집 방법과 관측성 수치화

- MCP 도구로 실시간 클러스터 데이터를 수집, AWS EKS · Document 서버를 통해 근거 확보
- Langfuse 트레이싱으로 토큰 누적 패턴·루프 횟수·비용을 정량적으로 분석 — 문제를 수치로 보게 됨

이게 맞나?

구조·일관성 개선· 토큰 72% 절감

- Strands Agent Skill 활용으로 재사용성을 가진 모듈 설계
- 지침 모호성 제거→ 판단 일관성 개선, 유효성 검증 로직 추가로 일관성 개선
- Message Caching 적용 및 cache_read 단가 1/10 활용 → 단일 실행 기준 72% 토큰 절감

기술 트렌드와 전망

기술 트렌드

2026~ 엔지니어링

Harness Engineering

AI가 자유롭게 판단하도록 내버려 두는 대신, 사람이 먼저 구조와 방향을 설계하고 AI는 그 안에서 실행하도록 통제하는 개발 방식

2026~ 방법론

AI-DLC

AI를 단순 코딩 도구가 아니라 요구사항부터 검증까지 개발 전 주기에 파트너로 참여시키는 구조화된 AI
구조화된 AI 협업 워크플로우

개인적인 생각 / 전망

Agent, 사람을 대체하는 게 아니라

우리의 경험을
공유하며
AI를
더 똑똑한 파트너로 진화

장애 판단 · 보안 승인 · 비용 결정 등

의사결정의 영역은 여전히 사람이 책임질 것 같습니다!

감사합니다

Thank you